

Relazione del progetto ING-Quiz

Daniele Greco

Matricola: 0001172676

`daniele.greco5@studio.unibo.it`

6 luglio 2026

Indice

1	Analisi	2
1.1	Descrizione e requisiti	2
1.2	Modello del Dominio	4
2	Design	7
2.1	Architettura	7
2.2	Design dettagliato	8
2.2.1	Gestione della sessione di gioco	8
2.2.2	Caricamento delle domande e gestione del fallback . . .	12
2.2.3	Gestione e persistenza della classifica	14
3	Sviluppo	16
3.1	Testing automatizzato	16
3.2	Note di sviluppo	18
4	Commenti finali	20
4.1	Autovalutazione e lavori futuri	20
4.2	Difficoltà incontrate e commenti per i docenti	21
A	Guida utente	22
B	Esercitazioni di laboratorio	24
B.1	daniele.greco5@studio.unibo.it	24

Capitolo 1

Analisi

1.1 Descrizione e requisiti

ING-Quiz è un'applicazione a quiz dedicata al mondo dell'informatica e delle tecnologie digitali, ispirata al format televisivo *Chi vuol essere milionario?*[5]. L'utente assume il ruolo di concorrente e affronta una sequenza di domande riguardanti argomenti quali programmazione, sistemi informatici, reti, hardware, software e cultura tecnologica generale. Ciascuna domanda è accompagnata da quattro possibili risposte, di cui una sola corretta. Lo scopo principale del gioco è rispondere correttamente a quindici domande. Una risposta corretta permette di avanzare alla domanda successiva e incrementa il punteggio del giocatore. Una risposta errata determina invece la conclusione della partita, salvo il caso in cui sia stato precedentemente attivato un aiuto che consente un secondo tentativo. Durante una partita, il giocatore può utilizzare tre differenti aiuti, ciascuno una sola volta:

- **50:50:** elimina due risposte errate dalla domanda corrente;
- **Double Chance:** consente al giocatore di effettuare un secondo tentativo nel caso in cui la prima risposta selezionata sia errata;
- **Switch:** sostituisce la domanda corrente con una domanda di riserva avente lo stesso livello di difficoltà.

L'applicazione mette a disposizione due modalità di gioco. Nella modalità normale, il risultato ottenuto dal giocatore può essere registrato nella classifica. Nella modalità allenamento, invece, il giocatore può utilizzare le medesime funzionalità senza che il punteggio venga memorizzato. La classifica associa a ciascun giocatore il miglior punteggio ottenuto. Se un giocatore conclude più partite utilizzando lo stesso nome, il risultato precedente viene

sostituito solamente quando il nuovo punteggio è maggiore. Il sistema deve inoltre permettere la consultazione delle regole del gioco, della classifica e delle informazioni necessarie per comprendere il funzionamento degli aiuti.

Requisiti funzionali

- Le domande devono riguardare argomenti inerenti all'informatica e alle tecnologie digitali.
- Il sistema deve permettere all'utente di inserire il proprio nome prima di iniziare una partita.
- Il sistema deve permettere di avviare una partita normale.
- Il sistema deve permettere di avviare una sessione di allenamento.
- Ogni partita deve essere composta da quindici domande da completare.
- Ogni domanda deve presentare quattro risposte, delle quali una sola corretta.
- Le domande devono essere suddivise in livelli di difficoltà crescente.
- Il sistema deve aggiornare il punteggio dopo ogni risposta corretta.
- Il sistema deve terminare la partita quando viene selezionata una risposta errata, salvo l'eventuale utilizzo dell'aiuto Double Chance.
- Il sistema deve considerare vinta la partita quando il giocatore risponde correttamente a tutte le quindici domande.
- Il giocatore deve poter utilizzare una sola volta ciascuno dei tre aiuti disponibili.
- L'aiuto 50:50 deve eliminare due risposte errate dalla domanda corrente.
- L'aiuto Double Chance deve consentire un secondo tentativo dopo una prima risposta errata.
- L'aiuto Switch deve sostituire la domanda corrente con una domanda di riserva dello stesso livello di difficoltà.
- Il sistema deve mantenere tre domande di riserva, una per ogni livello di difficoltà.

- Al termine di una partita normale, il sistema deve memorizzare il risultato del giocatore.
- Il sistema deve conservare un solo risultato per ciascun giocatore, corrispondente al miglior punteggio ottenuto.
- I risultati ottenuti nella modalità allenamento non devono essere registrati.
- L'utente deve poter visualizzare la classifica ordinata per punteggio decrescente.
- L'utente deve poter consultare le informazioni e le regole principali del gioco.
- Il sistema deve fornire un riscontro visivo e sonoro in caso di risposta corretta, risposta errata, vittoria e sconfitta.

Requisiti non funzionali

- L'applicazione deve poter essere eseguita sui principali sistemi operativi dotati di una Java Virtual Machine compatibile.
- L'interfaccia deve risultare semplice da utilizzare anche da parte di un utente che avvia il programma per la prima volta.
- Il sistema deve mantenere separata la logica del gioco dalla modalità con cui essa viene presentata all'utente.
- Un problema nel reperimento delle domande da una sorgente esterna non deve impedire necessariamente l'avvio di una partita, qualora siano disponibili domande alternative.
- I risultati della classifica devono rimanere disponibili tra esecuzioni successive dell'applicazione.
- L'assenza o l'impossibilità di riprodurre un effetto sonoro non deve compromettere il corretto svolgimento della partita.

1.2 Modello del Dominio

Il dominio di ING-Quiz ruota attorno allo svolgimento di una sessione di gioco, durante la quale un giocatore affronta una successione di domande

e accumula un punteggio. La *sessione di quiz* rappresenta l'insieme delle attività necessarie allo svolgimento di una singola partita. Essa comprende la selezione delle domande, la gestione della domanda corrente, la ricezione delle risposte e l'utilizzo degli aiuti disponibili. Una *partita* descrive l'avanzamento del giocatore nel quiz. Essa è caratterizzata da uno stato, da un punteggio e dalla posizione raggiunta nella sequenza delle domande. Prima dell'avvio la partita non è ancora iniziata; durante il gioco è in corso; al termine può concludersi con una vittoria oppure con una sconfitta. Ogni *domanda* appartiene a un livello di difficoltà e presenta esattamente quattro possibili *risposte*. Una sola risposta è corretta. La selezione della risposta corretta incrementa il punteggio e, salvo il raggiungimento dell'ultima domanda, permette di proseguire. Una risposta errata conclude normalmente la partita. Le domande sono distribuite su tre livelli di difficoltà crescente. La progressione della partita conduce quindi il giocatore da quesiti più semplici a quesiti più impegnativi. Oltre alle quindici domande previste per il normale svolgimento del quiz, sono presenti tre domande di riserva, una per ciascun livello di difficoltà. Durante la partita il giocatore dispone di tre *aiuti*. Ogni aiuto può essere utilizzato una sola volta e modifica temporaneamente il modo in cui viene affrontata la domanda corrente. Il 50:50 riduce le alternative disponibili, Double Chance consente un secondo tentativo e Switch sostituisce la domanda corrente con quella di riserva corrispondente. Il *giocatore* è identificato dal nome scelto all'inizio della sessione. Al termine di una partita normale, il punteggio ottenuto può concorrere alla *classifica*. Per ciascun giocatore viene mantenuto un solo risultato, corrispondente al miglior punteggio raggiunto. Ogni voce della classifica associa quindi un giocatore al suo record e al momento in cui esso è stato ottenuto. Le principali entità del dominio e le relazioni che intercorrono fra esse sono riassunte in Figura 1.1.

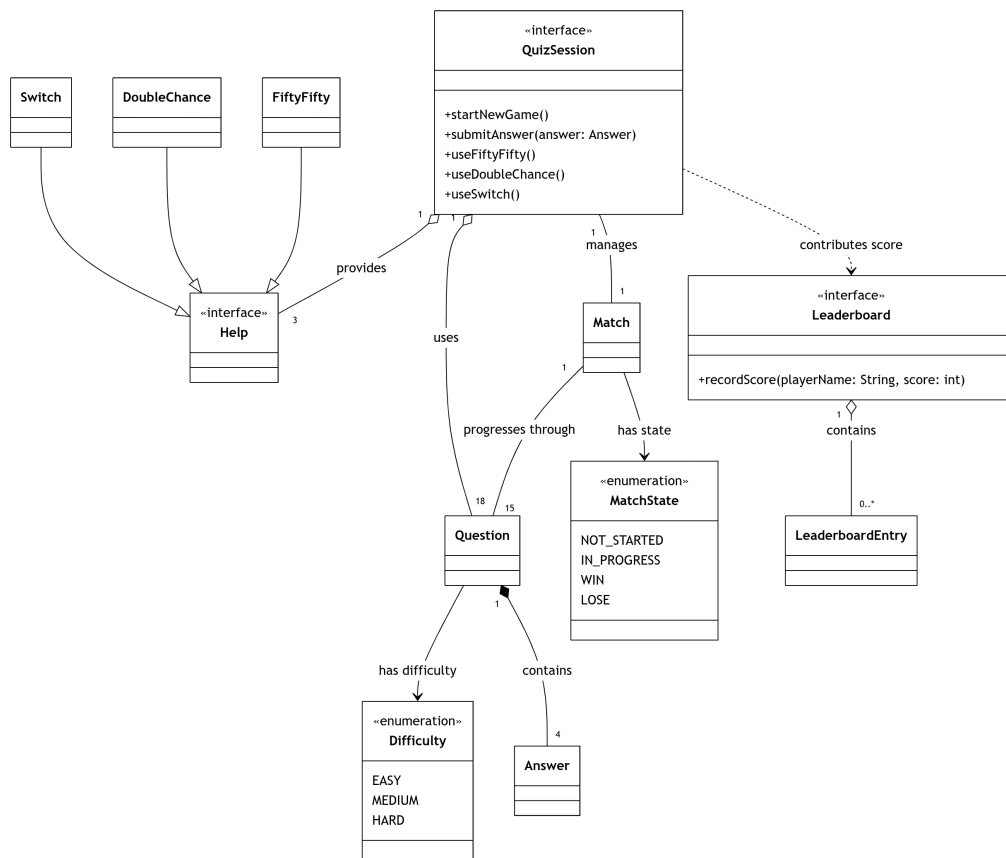


Figura 1.1: Modello del dominio di ING-Quiz.

Capitolo 2

Design

2.1 Architettura

L'applicazione è stata progettata seguendo il pattern architetturale Model–View–Controller. La suddivisione delle responsabilità consente di mantenere separata la logica del quiz dalla gestione dell'interfaccia grafica e dal coordinamento delle interazioni dell'utente. Il *Model* rappresenta il nucleo dell'applicazione e comprende le entità e le regole che governano lo svolgimento del quiz. I principali punti d'ingresso sono l'interfaccia **QuizSession**, utilizzata per gestire una singola sessione di gioco, e l'interfaccia **Leaderboard**, responsabile della gestione dei migliori punteggi. Il Model mantiene lo stato della partita, gestisce la progressione delle domande, valuta le risposte e applica il comportamento degli aiuti. Esso non dipende dalle componenti grafiche. La *View* espone tre interfacce principali: **QuizView**, **HomeView** e **GameView**. La prima rappresenta la finestra principale e consente di alternare la schermata iniziale e quella di gioco. Le altre due definiscono rispettivamente le operazioni relative alla home e alla partita. Le implementazioni concrete utilizzano un'interfaccia grafica desktop, ma i controller dipendono esclusivamente dalle relative astrazioni. Il *Controller* coordina le interazioni fra View e Model. La classe **QuizController** costruisce e collega i componenti principali dell'applicazione. **HomeController** gestisce l'avvio delle modalità di gioco, la consultazione della classifica e la chiusura dell'applicazione. **GameController** interpreta le azioni effettuate durante una partita, inoltrandole alla sessione attiva e aggiornando successivamente la View. **QuizSessionManager** mantiene il riferimento alla sessione corrente, al nome del giocatore e alla modalità selezionata. Le interazioni partono sempre dalla View, che notifica il Controller mediante funzioni registrate come listener. Il Controller invoca quindi le operazioni del Model e utilizza il risultato ottenuto per aggiorna-

re la View. Il Model non accede direttamente ai componenti grafici e non conosce il Controller. Questa separazione consente di sostituire l'implementazione grafica senza modificare la logica del quiz. Una diversa View dovrebbe implementare le interfacce già definite, mentre i controller e il Model potrebbero rimanere invariati. Analogamente, le sorgenti delle domande e la persistenza della classifica sono nascoste dietro apposite astrazioni, evitando che i dettagli di accesso ai dati si propaghino nel resto dell'applicazione. Le principali componenti architetturali e le loro dipendenze sono rappresentate in Figura 2.1.

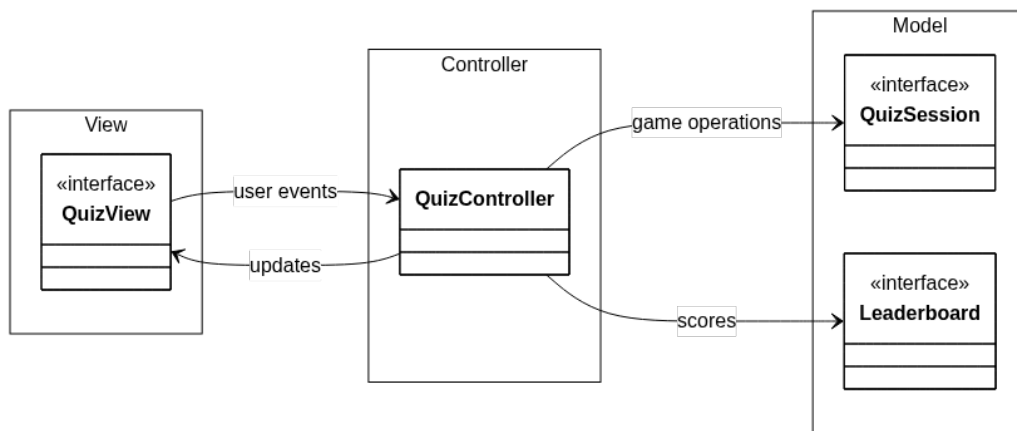


Figura 2.1: Architettura ad alto livello di ING-Quiz secondo il pattern MVC.

2.2 Design dettagliato

2.2.1 Gestione della sessione di gioco

Problema Lo svolgimento di una partita richiede il coordinamento di diversi elementi: la creazione della sessione, la selezione delle domande, l'avanzamento nel quiz, l'aggiornamento del punteggio, la gestione dello stato della partita e l'utilizzo degli aiuti. Esporre direttamente al Controller tutte le entità coinvolte avrebbe richiesto a quest'ultimo di conoscere numerosi dettagli della logica di gioco. Il Controller avrebbe dovuto, ad esempio, stabilire autonomamente quando incrementare il numero della domanda, quando dichiarare una vittoria o una sconfitta e come aggiornare lo stato degli aiuti. Una soluzione di questo tipo avrebbe prodotto un forte accoppiamento tra Controller e Model, rendendo più complesso modificare le regole del quiz senza intervenire anche sulla gestione dell'interfaccia.

Soluzione La gestione di una partita è stata concentrata dietro l'interfaccia `QuizSession`, che costituisce il principale punto di accesso alla logica del gioco. Il client può avviare una partita, ottenere la domanda corrente, inviare una risposta e utilizzare gli aiuti senza conoscere il modo in cui tali operazioni vengono realizzate internamente. L'implementazione `QuizSessionImpl` coordina le entità necessarie allo svolgimento del quiz. In particolare, essa mantiene l'insieme delle domande selezionate per la sessione, identifica la domanda corrente e delega a `Match` la gestione dello stato generale della partita. La classe `Match` conserva le informazioni relative al punteggio, alla posizione raggiunta e allo stato corrente. Le transizioni ammesse conducono la partita dallo stato iniziale allo stato in corso e, successivamente, a uno dei due stati terminali di vittoria o sconfitta. In questo modo la logica relativa allo stato della partita rimane separata dalla gestione concreta delle domande. La sessione applica inoltre le regole che determinano la progressione del gioco. In caso di risposta corretta, il punteggio viene incrementato e la partita prosegue, salvo che sia stata raggiunta la quindicesima domanda. In quest'ultimo caso la sessione conclude la partita con una vittoria senza tentare di accedere a una domanda successiva. Una risposta errata determina invece la sconfitta, a meno che sia attivo l'aiuto che consente un secondo tentativo. La creazione delle sessioni è delegata all'interfaccia `QuizSessionFactory`. In questo modo i componenti esterni non devono conoscere la classe concreta utilizzata né le dipendenze necessarie alla sua costruzione. Ogni nuova partita riceve quindi una nuova istanza di `QuizSession`, evitando di condividere accidentalmente lo stato tra partite successive. Le principali componenti coinvolte nella gestione della sessione sono rappresentate in Figura 2.2.

Pattern utilizzati L'interfaccia `QuizSession` svolge un ruolo assimilabile al pattern *Facade*. Essa offre al Controller un insieme ristretto e coerente di operazioni, nascondendo il coordinamento interno tra partita, domande, risposte e aiuti. La costruzione delle sessioni è invece affidata a una *Factory*. L'interfaccia `QuizSessionFactory` separa la richiesta di una nuova sessione dalla sua costruzione concreta. Questa scelta riduce l'accoppiamento tra il Controller e le implementazioni del Model e rende possibile modificare in futuro il processo di creazione senza cambiare i client della sessione. La gestione degli eventi della partita utilizza inoltre il pattern *Observer*. `Match` agisce come soggetto osservabile e produce eventi che descrivono i cambiamenti rilevanti, come l'avvio della partita, una risposta corretta, il passaggio alla domanda successiva, la vittoria o la sconfitta. Gli osservatori possono così reagire ai cambiamenti senza essere direttamente incorporati nella logica della partita. La gestione degli aiuti applica inoltre il pattern *Strategy*.

L'interfaccia generica `HelpStrategy<T>` definisce il comportamento comune degli aiuti, mentre `FiftyFifty`, `DoubleChance` e `Switch` ne forniscono implementazioni differenti. La sessione può quindi delegare l'applicazione dell'aiuto selezionato senza incorporarne direttamente la logica specifica.

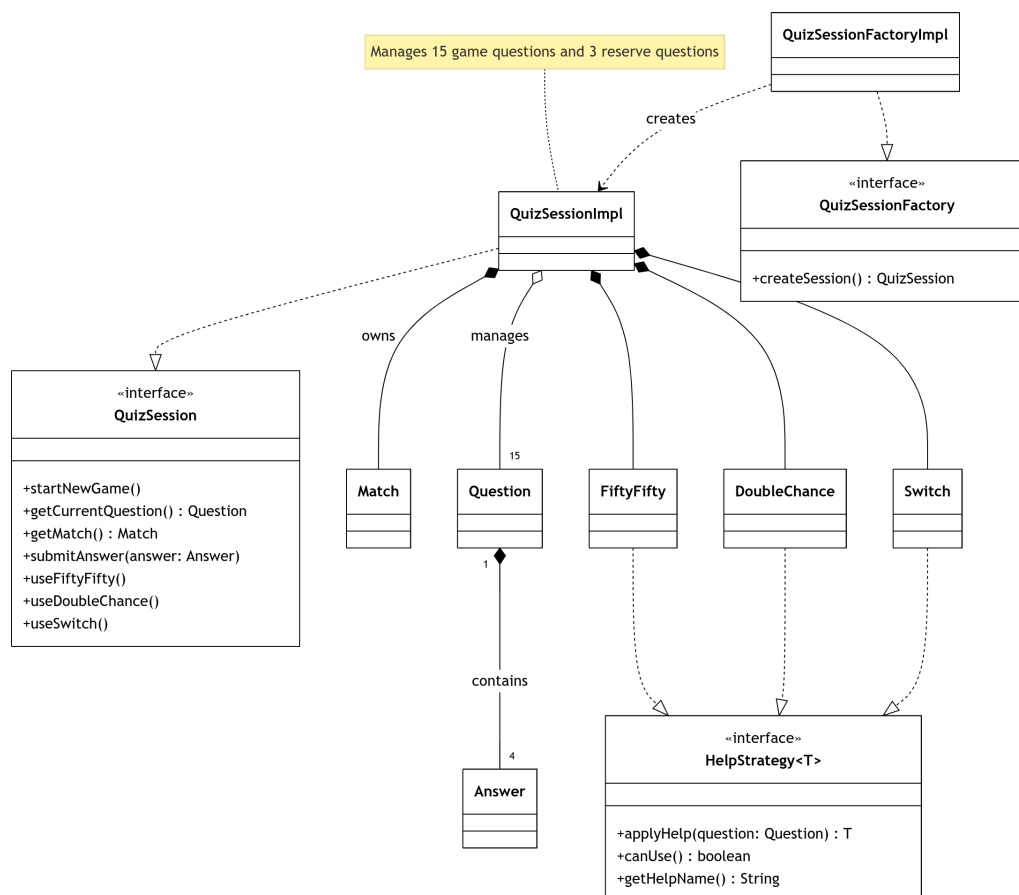


Figura 2.2: Struttura della sessione di gioco e delle componenti coordinate da QuizSessionImpl.

Come mostrato in Figura 2.2, `QuizSessionImpl` concentra il coordinamento delle principali entità coinvolte nello svolgimento della partita. Il Controller interagisce con il Model attraverso l'interfaccia `QuizSession`, senza dover conoscere la rappresentazione concreta della sessione né il modo in cui vengono gestiti partita, domande e aiuti. La presenza di `QuizSessionFactory` separa inoltre la costruzione delle sessioni dal loro utilizzo. Questa scelta consente di creare una nuova istanza per ogni partita e riduce la dipendenza dei client dall'implementazione concreta del Model.

Notifica degli eventi della partita

Problema Durante una partita si verificano cambiamenti rilevanti, come l'avvio del gioco, una risposta corretta, il passaggio alla domanda successiva, la vittoria o la sconfitta. Collegare direttamente la logica di **Match** ai componenti interessati a tali cambiamenti avrebbe aumentato l'accoppiamento e reso più difficile aggiungere nuovi comportamenti di reazione.

Soluzione La classe **Match** adotta il pattern *Observer*. Essa agisce come soggetto osservabile e consente la registrazione di osservatori interessati agli eventi della partita. Ogni cambiamento significativo viene rappresentato da un **MatchEvent**, che contiene il tipo di evento e lo stato aggiornato della partita. Gli osservatori dipendono dall'interfaccia generica **Observer<T>** e ricevono le notifiche senza che **Match** debba conoscerne l'implementazione concreta. Questa scelta rende possibile aggiungere nuovi osservatori senza modificare la logica della partita.

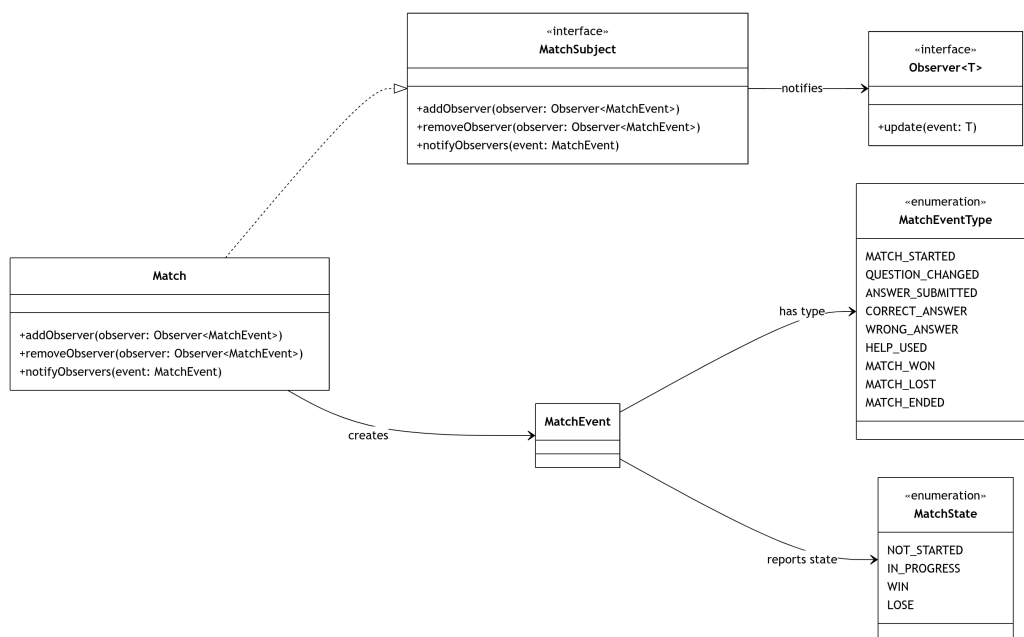


Figura 2.3: Applicazione del pattern Observer alla gestione degli eventi della partita.

Come mostrato in Figura 2.3, **Match** implementa l'interfaccia **MatchSubject** e mantiene una collezione di osservatori tipizzati su **MatchEvent**. Ogni evento descrive sia la natura del cambiamento sia lo stato aggiornato della partita,

permettendo agli osservatori di reagire senza introdurre dipendenze dirette verso la classe concreta `Match`.

2.2.2 Caricamento delle domande e gestione del fallback

Problema La sessione di gioco deve disporre di un insieme di domande coerente con la progressione prevista dal quiz. Il caricamento delle domande non può però essere legato a un'unica sorgente, poiché una sorgente remota può risultare temporaneamente non disponibile oppure restituire dati incompleti. Collegare direttamente la logica della sessione a una specifica modalità di caricamento avrebbe reso difficile sostituire la sorgente delle domande e avrebbe propagato nel Model dettagli estranei alle regole del gioco.

Soluzione L'accesso alle domande è stato astratto mediante l'interfaccia `QuestionDataRepository`. Le diverse modalità di caricamento implementano la stessa operazione e possono quindi essere utilizzate in modo intercambiabile. `RemoteQuestionDataRepository` recupera le domande da una sorgente remota e seleziona un numero prefissato di quesiti per ciascun livello di difficoltà. La risposta ricevuta viene interpretata da `TriviaParser`, che converte i dati esterni in oggetti di trasferimento indipendenti dalle entità del dominio. `LocalQuestionDataRepository` carica invece un insieme di domande disponibile localmente. Questa sorgente garantisce la possibilità di avviare una partita anche quando il caricamento remoto non può essere completato. Il coordinamento tra le due sorgenti è affidato a `FallbackQuestionDataRepository`. Esso tenta inizialmente il caricamento attraverso il repository principale e, in caso di errore, delega l'operazione al repository di riserva. Il resto dell'applicazione continua a dipendere esclusivamente da `QuestionDataRepository` e non deve quindi conoscere quale sorgente abbia effettivamente prodotto le domande. Prima di essere utilizzati nella partita, i dati caricati vengono convertiti nelle entità del dominio mediante `QuestionFactory`. La factory crea le domande e le relative risposte, assegna l'informazione di correttezza e normalizza il testo ricevuto dalla sorgente esterna.

Pattern utilizzati L'interfaccia `QuestionDataRepository` applica il pattern *Repository*, separando la logica del gioco dai dettagli relativi all'origine e al formato dei dati. L'aggiunta di una nuova sorgente richiede quindi una nuova implementazione dell'interfaccia, senza modificare i client che richiedono le domande. `FallbackQuestionDataRepository` svolge un ruolo

riconducibile al pattern *Decorator*. Esso implementa la medesima interfaccia dei repository che incapsula e arricchisce il comportamento dell'operazione di caricamento aggiungendo una strategia di recupero in caso di fallimento della sorgente principale. La costruzione delle entità del dominio a partire dai dati caricati è centralizzata in **QuestionFactory**. Questa scelta evita di distribuire la logica di conversione nei repository e mantiene separata la rappresentazione esterna delle domande da quella utilizzata dal Model.

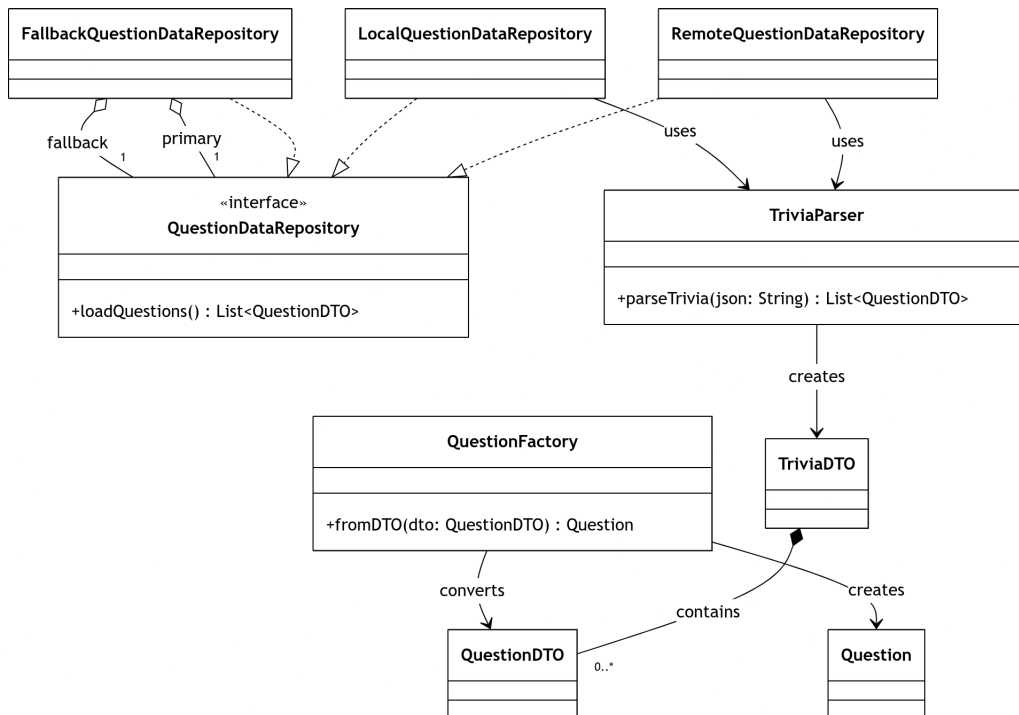


Figura 2.4: Struttura del caricamento delle domande e applicazione del meccanismo di fallback.

Come mostrato in Figura 2.4, tutte le sorgenti delle domande sono uniformate dall'interfaccia **QuestionDataRepository**. Il repository con fallback incapsula una sorgente principale e una di riserva, mantenendo trasparente al client quale delle due abbia completato il caricamento. Il parsing dei dati e la costruzione delle entità del dominio sono affidati a componenti distinti. In questo modo i repository si occupano del reperimento dei dati, **TriviaParser** della loro interpretazione e **QuestionFactory** della conversione nella rappresentazione utilizzata dal Model.

2.2.3 Gestione e persistenza della classifica

Problema Al termine di una partita normale, il risultato ottenuto dal giocatore deve poter essere conservato e successivamente mostrato nella classifica. La gestione della classifica deve inoltre applicare una regola precisa: per ogni nome deve essere mantenuto un solo risultato, corrispondente al miglior punteggio ottenuto. Concentrare nello stesso componente sia le regole di aggiornamento della classifica sia i dettagli relativi alla lettura e scrittura dei dati avrebbe reso il sistema più rigido e più difficile da estendere. Una modifica del formato di persistenza avrebbe infatti potuto influire direttamente sulla logica che stabilisce se un nuovo punteggio debba sostituire quello precedente.

Soluzione La gestione della classifica è esposta attraverso l'interfaccia **Leaderboard**. L'implementazione **LeaderboardImpl** contiene le regole che determinano l'inserimento e l'aggiornamento dei risultati. Quando viene registrato un punteggio, la classifica verifica se esiste già una voce associata allo stesso nome, confrontando i nomi senza distinguere fra lettere maiuscole e minuscole. Se il giocatore non è presente, viene creata una nuova voce. Se invece esiste già un risultato, esso viene sostituito solamente quando il nuovo punteggio è maggiore. Ogni risultato è rappresentato da **LeaderboardEntry**, che associa il nome del giocatore al miglior punteggio ottenuto e al momento in cui tale record è stato raggiunto. Le voci restituite dalla classifica vengono ordinate per punteggio decrescente e, in caso di parità, per data di conseguimento. La persistenza è separata dalla logica della classifica mediante l'interfaccia **LeaderboardRepository**. Essa espone le operazioni necessarie per caricare e salvare le voci, senza imporre uno specifico formato di memorizzazione. L'implementazione **JsonLeaderboardRepository** conserva i dati in un file esterno all'applicazione. Se il file non è ancora presente, la classifica viene considerata vuota; al primo salvataggio vengono create automaticamente le directory necessarie e il file contenente i risultati. Il salvataggio viene richiesto da **GameController** quando una partita normale termina con una vittoria o una sconfitta. Le sessioni di allenamento, invece, non producono alcun aggiornamento. La visualizzazione è gestita da **HomeController**, che recupera le voci ordinate e le inoltra alla View.

Pattern utilizzati Anche in questo caso viene utilizzato il pattern *Repository*. **LeaderboardRepository** separa le regole della classifica dai dettagli della persistenza. **LeaderboardImpl** dipende esclusivamente dall'interfaccia e può quindi essere utilizzata con una diversa modalità di salvataggio senza modificare la logica che confronta e ordina i punteggi. La distinzione fra

`Leaderboard` e `LeaderboardRepository` consente inoltre di mantenere separate due responsabilità: il primo componente decide quali risultati debbano essere conservati, mentre il secondo stabilisce come tali risultati vengano caricati e salvati.

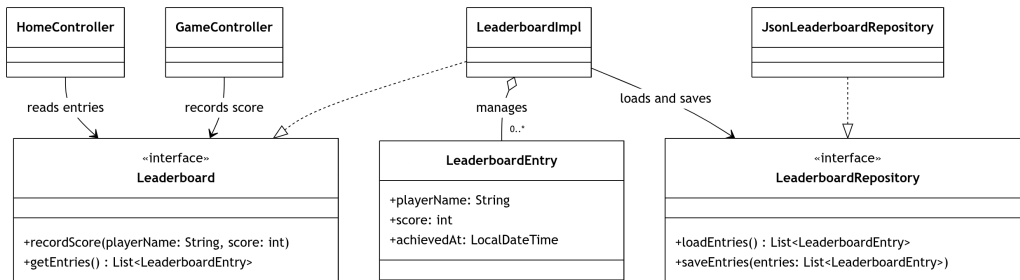


Figura 2.5: Separazione fra logica e persistenza nella gestione della classifica.

Come mostrato in Figura 2.5, `LeaderboardImpl` applica le regole di aggiornamento e ordinamento, mentre delega la persistenza a `LeaderboardRepository`. I controller dipendono dall'interfaccia `Leaderboard`: quello di gioco registra il risultato finale, mentre quello della schermata iniziale recupera le voci da mostrare. Questa suddivisione evita che i controller conoscano il formato del file e permette di sostituire `JsonLeaderboardRepository` con una diversa implementazione senza modificare la logica della classifica.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

Il progetto utilizza una suite di test automatizzati basata su JUnit 5. L'attività di testing si concentra principalmente sul Model e sui componenti di accesso ai dati, poiché queste aree contengono le regole del quiz e i comportamenti maggiormente soggetti a regressioni. I test sono organizzati seguendo la stessa suddivisione in package adottata nei sorgenti.

model.answer I test relativi alle risposte verificano la corretta costruzione delle entità, la conservazione del testo e dell'informazione che ne determina la correttezza. Vengono inoltre controllati i vincoli sui parametri forniti in fase di creazione.

model.question La parte dedicata alle domande verifica la corretta associazione fra testo, difficoltà e insieme delle risposte. I test controllano anche i principali vincoli strutturali dell'entità, impedendo la creazione di domande prive delle informazioni necessarie.

model.question.factory La factory delle domande viene testata separatamente per verificare la conversione dagli oggetti di trasferimento alle entità del dominio. In particolare, viene controllata la costruzione della risposta corretta e delle risposte errate, oltre alla normalizzazione delle entità HTML presenti nei testi ricevuti dalla sorgente remota.

model.match La parte più estesa della suite riguarda la gestione della partita e della sessione di quiz. I test di **Match** verificano lo stato iniziale, l'avvio

della partita, l'aggiornamento del punteggio, il passaggio alla domanda successiva e le transizioni terminali di vittoria e sconfitta. I test di `QuizSession` riproducono scenari completi, come l'avvio di una nuova partita, l'invio di risposte corrette o errate, l'avanzamento fra le domande e il raggiungimento della vittoria dopo la quindicesima risposta corretta. Le tre funzionalità di aiuto sono verificate in classi dedicate: vengono controllati l'eliminazione di due risposte errate, il secondo tentativo consentito da Double Chance e la sostituzione della domanda tramite Switch. Sono inoltre testati i vincoli che impediscono di utilizzare uno stesso aiuto più di una volta. Per isolare la sessione dalle sorgenti reali delle domande, questi test utilizzano implementazioni fittizie di `QuestionDataRepository`. Ciò permette di fornire insiemi di domande noti e di riprodurre in modo deterministico sia caricamenti validi sia condizioni di errore.

model.data.leaderboard La logica della classifica viene verificata mediante un repository in memoria. I test controllano l'inserimento di un nuovo giocatore, l'aggiornamento del record soltanto quando il nuovo punteggio è migliore, il confronto dei nomi senza distinzione fra maiuscole e minuscole e l'ordinamento decrescente delle voci. L'utilizzo di una persistenza fittizia evita che questi test dipendano dal file system.

data I componenti di caricamento e interpretazione delle domande vengono testati separatamente dal Model. Il repository locale viene verificato utilizzando risorse di test note, mentre il parser viene sottoposto sia a documenti validi sia a contenuti malformati o incompleti. Per il repository remoto viene utilizzata la libreria Mockito [3]. Essa consente di creare oggetti simulati per `HttpClient` e `HttpResponse`, controllandone il comportamento senza effettuare richieste reali verso il servizio esterno. In questo modo è possibile riprodurre in modo deterministico risposte HTTP valide, errori di comunicazione, codici di stato non corretti, body nulli o vuoti e interruzioni della richiesta. Il repository con fallback viene verificato controllando sia il normale utilizzo della sorgente principale sia l'attivazione della sorgente locale quando il caricamento remoto fallisce. Nel complesso, l'impiego di repository fittizi e mock consente di isolare le unità sotto test, evitare dipendenze dalla rete e dal file system e mantenere la suite completamente automatica e ripetibile.

3.2 Note di sviluppo

- **Definizione e utilizzo di tipi generici.**
Permalink: <https://github.com/dani3greco-u/00P25-ING-Quiz/blob/8c64b4fd3e04fa003cc8591a458a984550b486e9/src/main/java/it/unibo/common/Observer.java#L8C1-L17C2>
- **Utilizzo di Stream, lambda expression, method reference e Optional.**
Permalink: <https://github.com/dani3greco-u/00P25-ING-Quiz/blob/8c64b4fd3e04fa003cc8591a458a984550b486e9/src/main/java/it/unibo/model/data/leaderboard/LeaderboardImpl.java#L41-L72>
- **Utilizzo della Java HTTP Client API.**
Permalink: <https://github.com/dani3greco-u/00P25-ING-Quiz/blob/8c64b4fd3e04fa003cc8591a458a984550b486e9/src/main/java/it/unibo/data/question/RemoteQuestionDataRepository.java#L105-L152>
- **Parsing e serializzazione JSON tramite Jackson.**
Permalink: <https://github.com/dani3greco-u/00P25-ING-Quiz/blob/8c64b4fd3e04fa003cc8591a458a984550b486e9/src/main/java/it/unibo/data/question/TriviaParser.java#L38-L58>
- **Utilizzo di Apache Commons Text [4] per la gestione delle entità HTML.**
Permalink: <https://github.com/dani3greco-u/00P25-ING-Quiz/blob/8c64b4fd3e04fa003cc8591a458a984550b486e9/src/main/java/it/unibo/model/question/factory/QuestionFactory.java#L25-L41>
- **Utilizzo della Java Sound API.**
Permalink: <https://github.com/dani3greco-u/00P25-ING-Quiz/blob/8c64b4fd3e04fa003cc8591a458a984550b486e9/src/main/java/it/unibo/view/swing/audio/SoundPlayer.java#L27-L61>
- **Utilizzo di Mockito [3] per l'isolamento delle dipendenze nei test.**
Permalink: <https://github.com/dani3greco-u/00P25-ING-Quiz/blob/8c64b4fd3e04fa003cc8591a458a984550b486e9/src/test/java/it/unibo/data/RemoteQuestionRepositoryTest.java#L79-L95>

Scelta della sorgente principale delle domande Durante le prove dell'interfaccia grafica è emerso che la sorgente remota tendeva a restituire con una certa frequenza le stesse domande, riducendo la varietà percepita tra partite successive. Per questo motivo è stato deciso di utilizzare il repository locale come sorgente principale e quello remoto come sorgente di riserva. Il file locale è stato ampliato con un numero maggiore di domande suddivise per difficoltà. Questa scelta rende le partite più varie e, allo stesso tempo, evita che l'applicazione dipenda dalla disponibilità o dalla qualità della sorgente esterna.

Utilizzo di strumenti di intelligenza artificiale Durante lo sviluppo sono stati utilizzati strumenti di intelligenza artificiale come supporto alla comprensione e all'approfondimento di tecnologie non precedentemente affrontate in modo esteso, in particolare Jackson [2], Mockito [3] e Apache Commons Text [4]. Tali strumenti non sono stati impiegati per una copiatura passiva del codice. Ogni proposta è stata analizzata, confrontata con la documentazione delle librerie e adattata alla struttura del progetto. Prima dell'integrazione sono stati verificati il significato delle API utilizzate, il comportamento atteso e le conseguenze delle scelte progettuali. Il codice inserito nel progetto è stato quindi rielaborato e compreso, e il suo corretto funzionamento è stato verificato mediante test automatici e prove di integrazione.

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

Il progetto è stato sviluppato individualmente, occupandomi di tutte le fasi: analisi, progettazione, implementazione, testing e documentazione. Ritengo che il principale punto di forza del lavoro sia la struttura del Model. La logica della partita è stata separata dalla gestione dell'interfaccia grafica e dalle sorgenti dei dati, cercando di mantenere responsabilità ben definite e componenti sostituibili. Particolare attenzione è stata dedicata alla gestione della sessione, agli aiuti, al caricamento delle domande e alla persistenza della classifica. Un altro aspetto positivo è rappresentato dal testing automatizzato. Le parti più rilevanti del Model e del livello dati sono coperte da test che verificano sia i casi ordinari sia le principali condizioni di errore. L'impiego di repository fittizi e di mock ha permesso di isolare le componenti e rendere la suite ripetibile. Il progetto presenta tuttavia alcuni limiti. L'interfaccia grafica è funzionale ma potrebbe essere ulteriormente migliorata dal punto di vista estetico. La classifica è attualmente locale e non consente il confronto fra utenti su dispositivi differenti. Una possibile evoluzione sarebbe l'adozione di un database per la persistenza dei risultati e delle domande, soluzione che permetterebbe una gestione più strutturata dei dati e una futura estensione verso classifiche condivise. Tale sviluppo risulta oggi più realistico grazie alle competenze acquisite durante il percorso di studi. Tra i possibili sviluppi futuri rientra inoltre l'estensione delle categorie di gioco. L'applicazione è attualmente dedicata a domande di ambito informatico, ma l'architettura consente di introdurre una nuova schermata nella quale l'utente possa scegliere la categoria prima di iniziare la partita. In questo modo sarebbe possibile riutilizzare la stessa logica di sessione, mantenendo invariato il funzionamento generale del quiz e variando soltanto il tipo di domande

caricate.

4.2 Difficoltà incontrate e commenti per i docenti

La principale difficoltà incontrata durante lo sviluppo del progetto è stata la necessità di affrontare individualmente tutte le fasi del lavoro, a causa di circostanze esterne che hanno impedito la formazione di un gruppo. Questo ha richiesto una gestione particolarmente attenta del tempo e delle priorità. Allo stesso tempo, questa condizione ha reso il progetto un'occasione utile per seguire direttamente l'intero ciclo di sviluppo e comprendere meglio il legame fra le diverse fasi. In particolare, il corso ha contribuito in modo rilevante alla comprensione dell'importanza della progettazione, della separazione delle responsabilità e della costruzione di componenti riutilizzabili ed estendibili. Fra gli insegnamenti del percorso, questo corso è stato uno di quelli che ha fornito strumenti più concreti e immediatamente applicabili nello sviluppo di software strutturato. L'aspetto che ritengo più significativo è l'attenzione posta non soltanto sul funzionamento del codice, ma sul modo in cui esso viene organizzato, mantenuto e fatto evolvere. L'esperienza del progetto ha quindi consolidato competenze che vanno oltre la semplice implementazione e che considero utili anche per lavori futuri di maggiore complessità. Auspico inoltre che, nei corsi successivi, vi siano ulteriori occasioni per sviluppare in modo concreto la capacità di lavorare in gruppo. Ritengo infatti che la collaborazione, il coordinamento delle attività e il confronto fra più persone siano competenze fondamentali e tra le più richieste nel mondo del lavoro, in particolare nell'ambito dello sviluppo software.

Appendice A

Guida utente

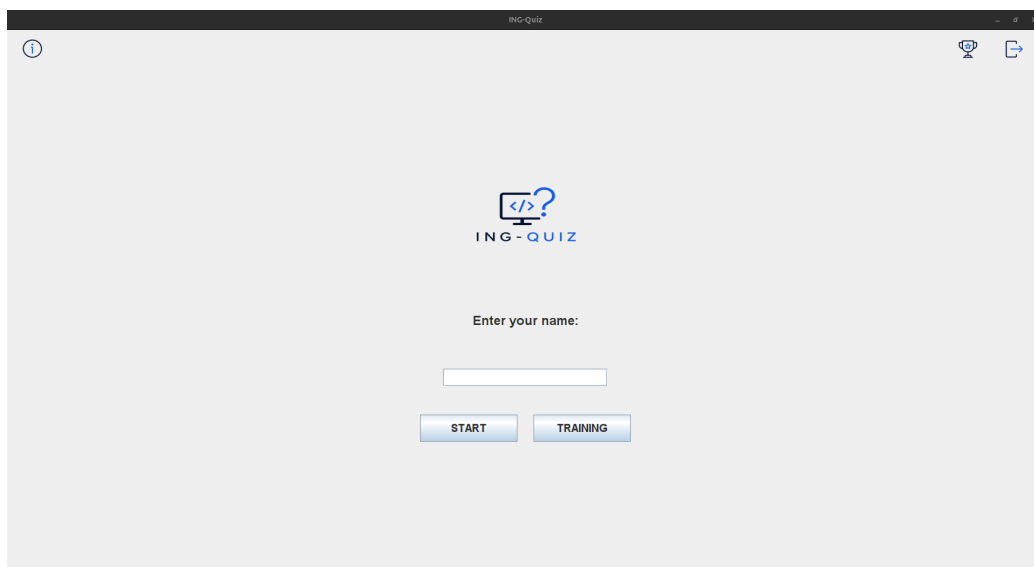


Figura A.1: Schermata Home

Per iniziare una partita:

- inserire il proprio nome nella schermata iniziale;
- scegliere la modalità di gioco desiderata;

Durante la partita è possibile utilizzare gli aiuti **50:50**, **Double Chance** e **Switch**, premendo i relativi pulsanti. Ciascun aiuto può essere utilizzato una sola volta per partita.

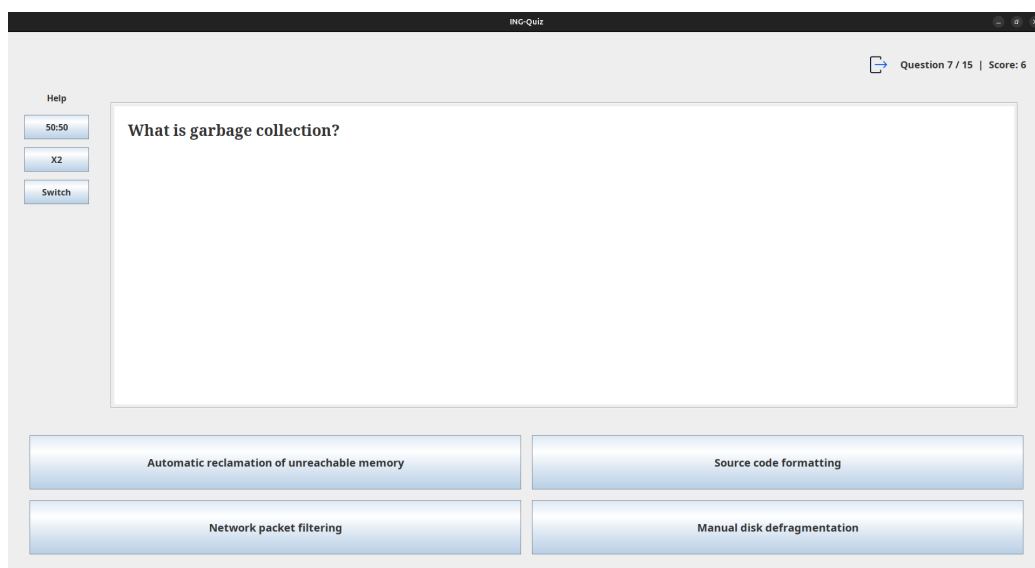


Figura A.2: Schermata di gioco

La classifica è salvata localmente sul computer dell'utente. Alla prima registrazione di un punteggio, l'applicazione crea automaticamente un file esterno al JAR, che viene poi aggiornato al termine delle successive partite in modalità normale. La classifica è accessibile dalla schermata iniziale e mostra il miglior punteggio registrato da ciascun giocatore.

Position	Player	Score	Date
1	daniele	5	26/06/2026 19:38
2	filippo	5	29/06/2026 15:10
3	antonio	1	29/06/2026 15:15

Figura A.3: Classifica dei giocatori

Appendice B

Esercitazioni di laboratorio

B.1 daniele.greco5@studio.unibo.it

- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287291>
- Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288319>
- Laboratorio 12: <https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290655>

Bibliografia

- [1] Open Trivia Database, *Open Trivia DB API*.
https://opentdb.com/api_config.php.
- [2] FasterXML, *Jackson Databind Documentation*.
<https://github.com/FasterXML/jackson-databind>.
- [3] Mockito, *Mockito Documentation*.
<https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockito.html>.
- [4] Apache Software Foundation, *Apache Commons Text Documentation*.
<https://commons.apache.org/proper/commons-text/>.
- [5] Wikipedia, *Chi vuol essere milionario?*.
https://it.wikipedia.org/wiki/Chi_vuol_essere_milionario%3F.